

# Troca de Chaves Híbrida no TLS 1.3: Combinando X25519 e ML-KEM

Day 02 | Category: Integração de Protocolos de Rede

Date: 2026-07-09 |

DAILY MONITORING

À medida que as organizações se preparam para a transição pós-quântica, substituir a criptografia de curva elíptica clássica (ECC) repentinamente traz riscos significativos. A criptografia baseada em reticulados (lattices) é relativamente nova em ambientes de produção comparada a padrões matemáticos de décadas atrás. Para fazer essa transição com segurança, a comunidade de criptografia padronizou os **Mecanismos de Encapsulamento de Chave Híbridos (Hybrid KEMs)**. O principal padrão de produção na internet hoje é o X25519MLKEM768, que combina o ECDH clássico em curva X25519 com o recém-padronizado ML-KEM-768 (FIPS 203).

**Princípio de Segurança Subjacente:** Um KEM híbrido combina dois mecanismos de troca de chaves independentes de modo que o segredo compartilhado resultante seja seguro desde que *peelo menos um* dos mecanismos constituintes permaneça seguro. Se uma falha sutil for descoberta no ML-KEM, o componente clássico X25519 impede a decifragem. Se um computador quântico viável for construído, o componente ML-KEM bloqueia o algoritmo de Shor.

## Como o X25519MLKEM768 Funciona no TLS 1.3

A integração da troca de chaves híbrida no protocolo Transport Layer Security (TLS) 1.3 opera dentro da estrutura padrão de handshake, introduzindo mudanças mínimas na máquina de estados, mas alterando os dados transmitidos:

- **Geração de Chaves no Cliente:** O cliente gera um par de chaves X25519 padrão e um par de chaves ML-KEM-768. Ele concatena as duas chaves públicas em uma única chave pública híbrida.
- **Client Hello:** Na extensão Supported Groups do ClientHello, o cliente lista o X25519MLKEM768 junto com os grupos herdados (como X25519). Na extensão KeyShare, envia a fração de chave pública híbrida concatenada, além de uma fração de chave pública X25519 autônoma como alternativa (fallback) para servidores antigos.
- **Encapsulamento e Resposta do Servidor:** Se o servidor suportar o grupo híbrido, ele executa o ECDH clássico na fração X25519 para gerar um segredo compartilhado clássico (\$K\_1\$) e executa o encapsulamento ML-KEM na fração ML-KEM-768 para gerar um texto cifrado (\$c\_2\$) e um segredo compartilhado pós-quântico (\$K\_2\$). Ele envia sua chave pública clássica e o texto cifrado pós-quântico no keyshare do ServerHello.

- **Combinador de Chaves (KDF):** Ambas as partes combinam os segredos clássico e quântico usando uma Função de Derivação de Chave (KDF) segura. Para evitar ataques ativos onde um adversário substitui um texto cifrado ou chave pública, o combinador vincula os dados:

`K = HKDF-Extract(Salt, K_1 || K_2 || c_1 || c_2 || pk_1 || pk_2)`

## Aplicabilidade na Cibersegurança e Status na Indústria (2026)

O principal motivador para a troca de chaves híbrida é a ameaça **Harvest Now, Decrypt Later (HNDL)** (Coletar Agora, Decifrar Depois). Adversários estão capturando ativamente tráfego cifrado da web hoje com a intenção de decifrá-lo no futuro assim que um Computador Quântico Relevante para Criptografia (CRQC) estiver disponível. Proteger a troca de chaves assimétrica hoje é crucial, pois qualquer dado transmitido agora é vulnerável a decifragem retrospectiva.

No início de 2026, a adoção do X25519MLKEM768 já domina a infraestrutura da internet:

- **Navegadores Web:** Google Chrome (desde a versão 131 em Nov de 2024) e Mozilla Firefox (desde a versão 132) habilitam essa troca híbrida por padrão para conexões HTTPS.
- **Redes de Distribuição de Conteúdo (CDNs):** Grandes CDNs como Cloudflare, Akamai e Google Edge processam uma enorme fatia do seu tráfego global TLS usando trocas de chaves híbridas, com a Cloudflare relatando que mais de 38% do tráfego HTTPS global usava PQC híbrido já no início de 2025.
- **Software Corporativo:** Em fevereiro de 2026, a Oracle integrou o JEP 527 no JDK 27, ativando suporte a trocas de chaves híbridas (como X25519MLKEM768 e SecP256r1MLKEM768) por padrão nos provedores JSSE do Java.

## Comparação de Algoritmos e Desempenho

| Algoritmo / Esquema   | Tipo             | Tamanho da Chave Pública (Bytes) | Tamanho do Texto Cifrado / Share (Bytes) | Base de Segurança Matemática         | Sobrecarga de Rede (Overhead)        |
|-----------------------|------------------|----------------------------------|------------------------------------------|--------------------------------------|--------------------------------------|
| X25519                | ECDH Clássico    | 32                               | 32                                       | Logaritmo Discreto em Curva Elíptica | Extremamente Baixa (~64 bytes total) |
| ML-KEM-768 (FIPS 203) | KEM Pós-Quântico | 1.184                            | 1.088                                    | Module Learning With Errors (M-LWE)  | Moderada (~2,2 KB total)             |

| Algoritmo / Esquema | Tipo        | Tamanho da Chave Pública (Bytes) | Tamanho do Texto Cifrado / Share (Bytes) | Base de Segurança Matemática    | Sobrecarga de Rede (Overhead)               |
|---------------------|-------------|----------------------------------|------------------------------------------|---------------------------------|---------------------------------------------|
| X25519MLKEM768      | KEM Híbrido | 1.216                            | 1.120                                    | M-LWE + Curvas Elípticas (ECDH) | Moderada (~2,3 KB total; bem dentro do MTU) |

## Quiz Diário: Teste seu Conhecimento

1. Por que um KEM híbrido como o X25519MLKEM768 é preferido em vez de uma implementação pura de ML-KEM-768 durante o período de migração?

A) Porque o ML-KEM-768 é muito lento para rodar em dispositivos móveis modernos sem aceleração por hardware.

B) Para mitigar o risco de falhas matemáticas desconhecidas ou recém-descobertas na criptografia baseada em reticulados, enquanto ainda se defende contra futuros computadores quânticos.

C) Porque o ML-KEM-768 não suporta encapsulamento de chave, podendo ser usado apenas para assinaturas digitais.

**Resposta Correta: B.** A abordagem híbrida garante que a segurança seja mantida se *qualquer um* dos dois componentes permanecer seguro, evitando uma falha de segurança catastrófica se ocorrer uma descoberta criptanalítica na teoria de reticulados.

2. Como o cliente TLS 1.3 evita falhas de conexão ao se conectar a um servidor antigo que não suporta troca de chaves híbrida?

A) Ele rebaixa a conexão para HTTP em texto claro automaticamente.

B) Enviando uma fração de chave pública X25519 clássica autônoma na extensão KeyShare do ClientHello, junto com a fração híbrida.

C) Usando o ML-KEM-768 como um algoritmo de assinatura para assinar o handshake.

**Resposta Correta: B.** Ao incluir um keyshare X25519 clássico além do híbrido na extensão KeyShare, servidores legados podem simplesmente ignorar a fração híbrida e realizar um handshake ECDH padrão.

3. Qual é a finalidade de incluir as chaves públicas e textos cifrados no combinador KDF de um KEM híbrido?

A) Aumentar o tamanho do segredo compartilhado para evitar força bruta.

B) Garantir que a KDF seja totalmente compatível com blocos de tamanho SHA-256.

C) Vincular o segredo derivado às chaves públicas e textos cifrados específicos, impedindo que atacantes ativos substituam componentes da troca híbrida.

**Resposta Correta: C.** Se a KDF combinasse apenas os segredos (ex:  $H(K_1 || K_2)$ ), um atacante que quebrasse um dos esquemas poderia substituir o texto cifrado e forçar a chave final sem quebrar o outro esquema. Vincular chaves e textos cifrados na KDF previne esse ataque de substituição.